

动态规划入门 背包类问题

主讲：黄凤林

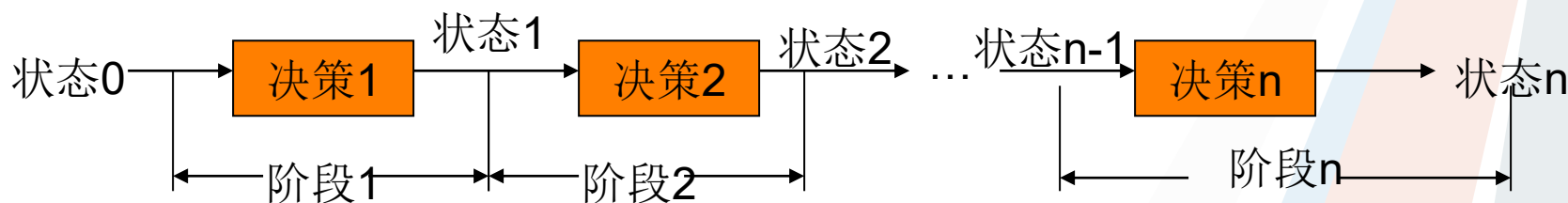


→ 动态规划

- **动态规划**(Dynamic Programming)是**研究多阶段决策过程最优化问题的一种数学方法**，英文缩写DP。
- 在动态规划中，**为了寻找一个问题的最优解（即最优决策过程），将整个问题的划分成若干个相应的阶段，并在每个阶段都根据先前所作出的决策作出当前阶段最优决策，进而得出整个问题的最优解。**
- 能采用动态规划求解的问题的一般要具有三个性质：
- **最优化原理**：如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优化原理。
- **无后效性**：即某阶段状态一旦确定，就不受这个状态以后决策的影响。也就是说，某状态以后的过程不会影响以前的状态，只与当前状态有关。
- **有重叠子问题**：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划适用的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）

动态规划

在现实生活中，有一类活动的过程，由于它的特殊性，可将过程分成若干个相互联系阶段，在它的每一个阶段都需要作出决策，从而使整个问题达到最好的活动效果。当然各个阶段决策的选取不是任何确定的，它依赖于当前面临的状态，又影响以后的发展，各个阶段决策确定后，就组成一个决策序列，因而也就确定了整个过程一条活动路线，这种把一个问题看作是一个前后关联具有链状结构的多阶段过程就称为多阶段决策过程，这种问题就称为多阶段决策问题。如下图：





→ 动态规划

- ◆ 动态规划是求解多阶段决策性最优解问题的有利武器，对思维难度要求高，常见实现策略**递推和记忆化搜索**两种，唯有多练习题目才能提高。
- ◆ **动态规划根据元素间关系，可以划分为常见模型：**
- ◆ 线性动态规划（一维、二维）
- ◆ 坐标动态规划（二维、三维）
- ◆ 区间动态规划（区间范围内）
- ◆ 背包动态规划（9种背包）
- ◆ 树形动态规划（基于树上的记忆化搜索）
- ◆ 状压动态规划（数位、按位DP）
- ◆ **动态规划的优化（单调性、斜率、四边形）**

➔ 背包类问题

- **背包问题：**是一类常见的选择问题。问题大概意思是：有一个体积为 V 的背包，现有 n 种物品可以选择。每种物品有一个体积 v_i 和一个价值 w_i 。问：怎么选择物品才可以在背包体积范围内让物品价值最大。
- **问题特点：**元素(物品)之间的关系是离散的，不存在元首之间前后依赖关系。**仅存在放进集合内，还是不放在集合内（也就是常说的选还是不选）。**
- **问题扩展：**物品之间增加主从关系、物品选择数量限制等？

→ 背包问题

背包问题是一类常见的选择问题。问题大概意思是：有一个体积为 V 的背包，现有 n 种物品可以选择。每种物品有一个体积 v_i 和一个价值 w_i 。问：怎么选择物品才可以在背包体积范围内让物品价值最大。

很多同学想的是，先拿性价比最高的(w_i/v_i)，再逐个拿性价比低一点的。这种思考方法在某些情况下是可以的，比如这种物品可以细分的时候，比如水，沙子。**但是实际情况中，很多物品是不可以分开的。这种情况下，这种解题思路就是错误的。比如**

$V=10$;
 $v_1=5$ $w_1=4$
 $v_2=5$ $w_2=4$
 $v_3=6$ $w_3=7$

虽然第三种物品的性价比最高，但是由于体积的关系，取一、二两种物品的总价值更大。



→ 01背包

有 N 件物品和一个容量为 V 的背包。第 i 件物品的体积是 $c[i]$ ，价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的体积总和不超过背包容量，且价值总和最大。

注：每个物品最多选取一件

01背包特点：**每种物品有且仅有一件，并且不可以再分**。该物品要么取走，要么不取，只有两种状态，所以叫01背包。同时，01背包也是所有背包模型的基础。

输入：

10 4

2 1

3 3

4 5

7 9

输出：

12

$V \leq 5000, N \leq 5000.$

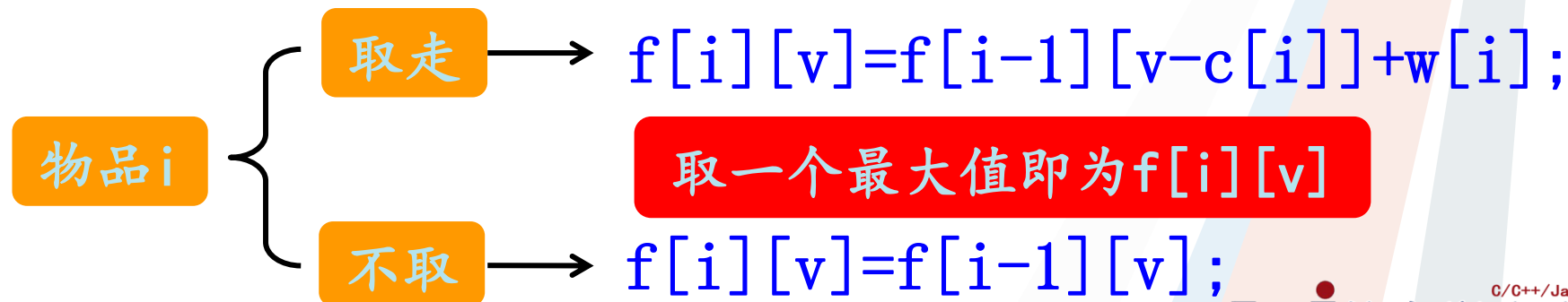
→ 解题思路

思路一：每一种物品有两种状态（0和1），判断所有组成的方案，然后找出其中价值最大的。时间复杂度 $O(2^n)$ 。理论上是不可能实现的。

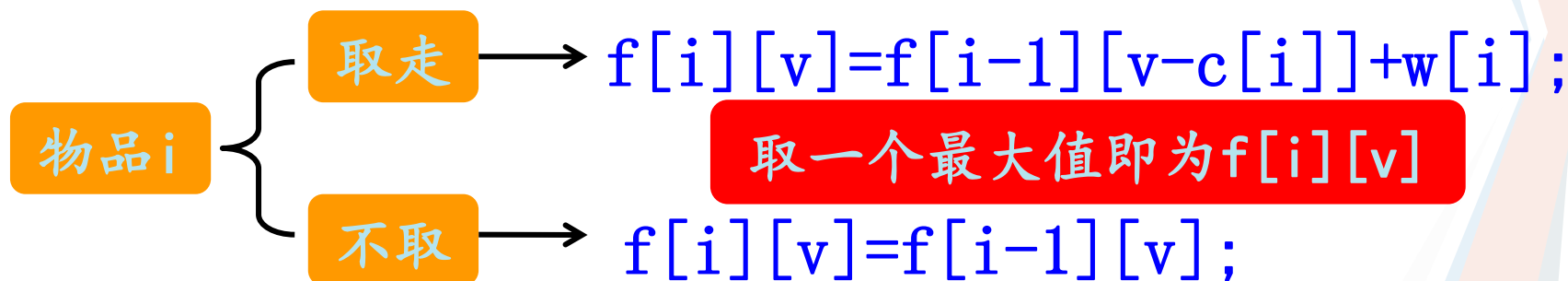
思路二：回溯法。时间复杂度和上面差不多。

思路三：递推

每种物品只有两种状态（0和1）。要求背包的最大价值。设前 i 件物品（部分/全部）放入体积为 v 的背包中的最大价值是 $f[i][v]$ 。



01背包



取走： $f[i][v] = f[i-1][v-c[i]] + w[i];$

如果要把第 i 件物品放入背包中，说明背包中肯定空余第 i 件物品体积的位置。说明前 $(i-1)$ 件物品占的体积为 $v-c[i]$ ，那么最大价值就是 $f[i-1][v-c[i]]$ 。既然把第 i 件物品放进去了，最大价值就 $+w[i]$ 。

不取： $f[i][v] = f[i-1][v];$

如果第 i 件物品不放入背包中，说明第 i 件物品是否存在对结果没有影响。前 $i-1$ 件物品占用的最大体积就是 v 。最大价值就是 $f[i-1][v]$ 。

$$f[i][v] = \max(f[i-1][v-c[i]] + w[i], f[i-1][v]);$$



→ 01背包

$f[i][v] = \max(f[i-1][v-c[i]] + w[i], f[i-1][v]);$

很明显，递推公式是一个二维数组，所以这是一个二维递推

```
for(int i=1;i<=n;i++)//枚举所有物品
```

```
{  
    for(int v=0,v<=V,v++)//枚举范围内所有体积
```

```
{  
    if(c[i]<=v)//如果该物品可以放进去
```

```
{  
        f[i][v]=max(f[i-1][v-c[i]]+w[i], f[i-1][v]);
```

```
} else //如果该物品肯定放不进去
```

```
    f[i][v]=f[i-1][v];
```

```
}
```

```
}
```



→ 01背包

$i-1$

		$v-c[i]$					
--	--	----------	--	--	--	--	--

i

					v		
--	--	--	--	--	-----	--	--

对于前 i 件物品，他的递推公式都是通过前 $i-1$ 件物品推导过来的，所以 v 的范围无论是 $[0, V]$ 还是 $[V, 0]$ 都没有任何关系。

递推肯定要初始化，即 $i=0$ 的时候的 $f[0][v]$ 的值就是0，同样的，当 $v=0$ 的时候， $f[i][0]$ 的值也仍然为0. 所以不需要额外的初始化。

我们所求的 n 件物品放入 V 的背包中的最大价值就是 $f[N][V]$ 的值



→ 01背包

时间复杂度: $O(V*N)$

空间复杂度: $V*N$

当 V 和 N 比较大的时候。在不超时的情况下，可能会超空间。所以我们需要优化一下。在做的时候我们已经发现了，第 i 行的值至于第 $i-1$ 行有关，所以我们可以滚动求值。 $j=i\%2$ 。只需要开一个 $f[2][V]$ 的数组就可以了。

实际上一维数组就可以了。

```
for(int i=1;i<=n;i++){  
    for(int v=m, v>=w[i], v--){  
        {  
            f[v]=max(f[v-c[i]]+w[i], f[v]);  
        }  
    }  
}
```

如果用一维数组表示，背包体积的枚举只能 $[v, w[i]]$ ，不能反过来思考为什么？



→ 01背包

有 N 件物品和一个容量为 V 的背包。第 i 件物品的体积是 $c[i]$ ，价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的体积**恰好**等于背包容量，且价值总和最大。问最大价值是多少

输入：

10 3

4 4

5 6

6 3

输出：

7

$N \leq 2000$ ， $V \leq 20000$.



➔ 多重背包

有N种物品和一个容量为V的背包。每种物品都有一个体积和价值，以及该物品有多少件。求解怎么装可使这些物品的体积总和不超过背包容量，且价值总和最大。问最大价值是多少

输入：

10 4

3 4 3

4 5 3

4 3 1

5 4 4

输出：

13

$N \leq 2000$, $V \leq 20000$.

→ 解题思路

这种背包问题和前面的背包问题不同的地方在于，一种物品可以有很多件，并不是只可以取一件。我们可以把它转换成01背包来求解。每种物品有 $m[i]$ 件，可以理解为有 $m[i]$ 种该物品，每种物品只有一件。这就转换成了01背包。

```
for(int i=1;i<=n;i++)//n种物品
{
    for(int j=1;j<=m[i];j++)//每种物品的数量
    {
        for(int v=m, v>=j*w[i], v--)//背包体积
        {
            f[v]=max(f[v-j*c[i]]+j*w[i], f[v]);
        }
    }
}
```

也可以直接开一个数组把所有物品当做一个物品来存储。再用01背包来做。

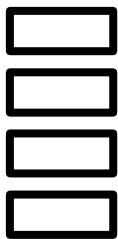


→ 优化方法

根据代码可知，该题的时间复杂度为 $O(V*N)$ （ N 表示所有物品的总数量），如果每一种物品的数量比较多，该题就容易超时，所以我们需要想办法优化一下。

对于第 i 种物品，他的数量为 $m[i]$ ，前面的做法是一个一个的枚举。即需要把 $0--m[i]$ 的所有可能性都表示出来。其实我们可以用更少的数就把 $0--m[i]$ 表示出来。

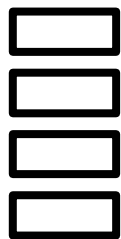
有一种答题卡，从上到下只有4个方格，但是可以选择的答案有十种。涂的时候可以涂多个方格，从上到下的方格填哪些数字数字才可以完整的表示出这十种可能。



用4个方格表示出1-10.



→ 优化方法



用4个方格表示出1-10。
每个方格表示一个整数。

答案是：1 2 3 4

根据上面的情况，我们可以发现，1-10可以通过1, 2, 3, 4的组合来表示。所以对于任意一个数 n 。我们都可以用若干个更少的数字组合表示出来，那最少的个数是多少个呢？我们发现可以用1, 2, 4, 8... 这样 $\text{ceil}(\log n)$ 个数来表示，个数是最少的。这样子就把 n 的枚举范围缩小到 $\text{ceil}(\log n)$

同样的，该题我们也可以用这种方法把每种 m 个物品变为每种物品 $\lg m$ 个。计算的时候时间复杂度就下降了。

比如17

1 2 4 8 2



→ 二进制优化

把每一种物品的的 m 个转换成 $1, 2, 4, \dots$ 这样的2的倍数个。
注意不能增加他能表示的数的范围。同时转换的时候要注意，体积和价值都要同时乘以倍数。

比如对于某种物品 3 4 9

可以转换成

3 4 1

6 8 2

12 16 4

6 8 2

这样四种物品。

这种二进制优化办法优化效率并不是特别高。特别是当 N 比较大， $m[i]$ 表示小的时候。优化效率有限。

还有另外一种优化办法——单调队列优化。



→ 完全背包

有 N 种物品和一个容量为 V 的背包。每种物品可以取无限次，每种物品有一个体积和价值。求解怎么装可使这些物品的体积总和不超过背包容量，且价值总和最大。

完全背包特点：每一种物品都可以取无限次且不可分割。

输入：	输出：
10 4	12
2 1	
3 3	
4 5	
7 9	

$N \leq 2000, V \leq 2000;$



→ 解题思路

和前面的多重背包一样，也可以转换成01背包来求解。虽然物品有无限件，但是由于背包的体积有限，每种物品最多可以放的数量是有限的，即最多放 $V/c[i]$ 件。这就转换成了多重背包，然后再转换成01背包即可。

```
for(int i=1;i<=n;i++) //n种物品
{
    for(int j=1;j<=V/c[i];j++) //每种物品的数量
    {
        for(int v=m; v>=j*w[i]; v--) //背包体积
        {
            f[v]=max(f[v-j*c[i]]+j*w[i], f[v]);
        }
    }
}
```



→ 优化方法

优化办法一：因为物品有无限件，所以对于任意的两种物品，如果 $w[i] < w[j]$ 并且 $c[i] > c[j]$ ，那么就说明 i 这种物品是肯定不可以取的。这样就可以筛选出很多不合理的物品。

优化办法二：

```
for(int i=1; i<=n; i++)  
{  
    for(int v=V; v>=c[i]; v--)  
    {  
        f[v]=max(f[v-c[i]]+w[i], f[v]);  
    }  
}
```



→ 优化方法

完全背包的写法和01背包的写法有点类似，只是内层循环的时候，01背包是 $[V, 0]$ ，而完全背包是 $[0, V]$ 。前面我们已经探讨过了。如果用一维数组来表示01背包，内循环只能是 $[V, 0]$ ，因为对于当物品种类为 i 的时候 v 的值，所需要的 $f[v - c[i]]$ 的值实际上是当物品种类为 $i-1$ 的时候的未更新的值，因为 $v > v - c[i]$ 。如果循环写出 $[0, V]$ ，那么第一次更新了 $f[v_1]$ 的值，第二次求 $f[v_2]$ 的的值的时候，假设 $v_2 - c[i] = v_1$ 。实际上是已经取过一次第 i 件物品的时候的 $f[v_1]$ 的值了，这和01背包不相符合。但是这刚好符合完全背包的要求。没意见物品可以取多次，直到背包放不下为止。



→ 优化方法

内循环的范围是 $[0, V]$, 所以我们求值的顺序是 $v_1-v_2-v_3-v_4$.

$f[v_1]$: 取第 i 件物品一次时的值. ($f[v_1]$ 值已经更新)

$v_2 = v_1 + c[i]; f[v_2] = \max(f[v_2 - c[i]] + w[i], f[v_2])$

$= \max(f[v_1] + w[i], f[v_2])$, 因为在计算 $f[v_2]$ 之前就已经计算过 $f[v_1]$ 了, 所以该表达式表示的是取第 i 件物品两次时的价值

$v_3 = v_2 + c[i]; f[v_3] = \max(f[v_3 - c[i]] + w[i], f[v_3])$
 $= \max(f[v_2] + w[i], f[v_3]);$

在计算 $f[v_3]$ 之前就已经计算过 $f[v_2]$ 了, 所以 $f[v_2]$ 表示取两次第 i 件物品的时候的最大价值, 那么该表达式就是表示取取第 i 件物品三次时的价值

$v_4 = v_3 + c[i];$ 也是也前面相同。

所以, 对于每一个 i , 随着 v 值的不断增加, 实际上里面就包含了取第 i 件物品的时候的所有情况了。

所以直接 $O(V*N)$ 就搞定了。

混合背包

有 n 种物品，里面有的物品只可以取一次，有的物品可以取 m 次，有的物品可以取无限次。求最大价值是多少

经过前面的学习，我们会发现，多重背包可以用二进制优化直接转换成01背包。写法完全一样。所以当只含有多重背包和01背包时直接转换成01背包求解。但是完全背包写法和01背包是相反的。所以就分情况讨论。

多重背包变01背包。

```
for(int i=1;i<=n;i++)  
{  
    if(01背包)  
        01背包求解[V, 0];  
    else if(完全背包)  
        完全背包求解[0, V];  
}
```




→ 二维背包

有 N 件物品和一个容量为 V ，承重量为 M 的背包。第 i 件物品的体积是 $c[i]$ ，重量是 $z[i]$ ，价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的体积总和不超过背包容量，并且重量在背包的承重范围之内，且价值总和最大。

$f[i][v][m]$ 表示当背包体积为 v ，承重量为 m ，选择前 i 件物品放入时的最大价值。

按照普通01背包的推导方法。第 i 件物品可以取可以不取。

如果取第 i 件物品

$$f[i][v][m] = f[i-1][v-c[i]][m-z[i]];$$

如果不取第 i 件物品

$$f[i][v][m] = f[i-1][v][m];$$

然后取最大值即可。

→ 二维背包

和前面01背包一样，可以化三维为二维，只需要枚举物品种类，背包体积，背包承重量就可以了。

```
for(int i=1;i<=n;i++)//物品种类
{
    for(int v=V;v>=c[i];v--)//背包体积
    {
        for(int m=M;m>=z[i];m--)//背包承重量
        {
            f[v][m]=max(f[v-c[i]][m-z[i]]+w[i], f[v][m]);
        }
    }
}
```

如果该题限制条件更多，取物品的方式也更多，只需要增加循环和前面的混合背包就可以求解了



➔ 分组背包

有 N 件物品和一个容量为 V 的背包。第 i 件物品的费用是 $c[i]$ ，价值是 $w[i]$ 。这些物品被划分为若干组，每组中的物品互相冲突，最多选一件。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量，且价值总和最大。

输入：

10 6 3

2 1 1

3 3 1

4 8 2

6 9 2

2 8 3

3 9 3

输出：

20

第一行输入依次是背包容量，物品种类以及物品组数。
接下来每行依次是体积，价值和所属组数。



➔ 分组背包

对于分组背包问题，我们还是可以看做01背包来求解，01背包是对于每种物品，有取和不取两种状态，而对于分组背包而言，是对于**每一组**物品，也只有两种状态，取走该组内其中一个和一个都不取(每组物品相互冲突)。所以分组背包的每一组仍然可以看做01背包求解。

很明显，循环有三层，一层是物品的组数，一层是该组物品的个数，一层是背包的容量。要确定好循环的嵌套顺序。注意，每组内的物品最多只能选择其中一个，所以一对于确定的一个背包容量和确定的物品组数，需要遍历该组内所有的物品，并且从中选择一个(不选)来得到一个最大值。



➔ 分组背包

```
for(int k=1;k<=t;k++)//物品的组数
{
    for(int v=V;v>=0;v--)//背包容量
    {
        for(int i=1;i<=a[k][0];i++) {
            //a[k][0]第k组物品的总量
            //当背包容量和组数确定后，我们是从该组物品里面
            //选择一个让当前的最大价值最大。
            if(v>=c[a[k][i]])//a[k][1]表示第k组物品第i个
            {
                int tmp=a[k][i];
                f[v]=max(f[v], f[v-c[tmp]]+w[tmp]);
            }
        }
    }
}
```



➔ 有依赖的背包

金明今天很开心，家里购置的新房就要领钥匙了，新房里有一间金明自己专用的很宽敞的房间。更让他高兴的是，妈妈昨天对他说：“你的房间需要购买哪些物品，怎么布置，你说了算，只要不超过N元钱就行”。今天一早，金明就开始做预算了，他把想买的物品分为两类：主件与附件，附件是从属于某个主件的。如果要买归类为附件的物品，必须先买该附件所属的主件。每个主件可以有0个、1个或2个附件。附件不再有从属于自己的附件。金明想买的东西很多，肯定会超过妈妈限定的N元。于是，他把每件物品规定了一个重要度，分为5等：用整数1~5表示，第5等最重要。他还从因特网上查到了每件物品的价格（都是10元的整数倍）。他希望在不超过N元（可以等于N元）的前提下，使每件物品的价格与重要度的乘积的总和最大。设第j件物品的价格为 $v[j]$ ，重要度为 $w[j]$ ，共选中了k件物品，编号依次为 $j_1, j_2, \dots, j_k$ ，则所求的总和为：
 $v[j_1]*w[j_1]+v[j_2]*w[j_2]+ \dots +v[j_k]*w[j_k]$ 。（其中*为乘号）
请你帮助金明设计一个满足要求的购物单。



有依赖的背包

有依赖的背包是指，一些物品必须依赖于另一些物品而存在，我们把他们称为主件和附件。如果要买附近，就一定要买对应的主件，反之，如果要买主件，不一定必须要买其附件。

该题要求比较简单，一个附件只从属于一个主件，一个主件最多只有两个附件。我们还是可以用01背包来考虑。先根据主件做01背包。然后只有五种情况（一个都不选择，只选择主件，选择主件和附件1，选择主件和附件2，选择主件和附件1, 2），然后选择五种情况中的最大值。但是一定要注意必须先满足在背包的容量范围之内

。还有一些更加复杂的有依赖的背包问题后面遇到再做讲解。



泛化背包

对于某些物品而言，他没有固定的费用和价值，他的价值是随着你分配的费用而发生着变化。比如你分配的费用较多，他的单个价值就变大。你分配的费用较少，他的单个价值就变小。他的价值不是固定的。这类背包问题也可以看做完全背包来求解。即该类物品理论上有限种，但是考虑到背包的体积他最多只可以放 $V/c[i]$ 个。解题办法和不优化的完全背包类似。尝试该物品所有可以放的可能性然后从中得到最优的解决办法。



背包方案总数

给你很多 n 种面值的货币，求组成面值为 m 的货币有多少种方案。

输入：

3 10

1

2

5

输出：

10

样例解释：有3种面值的货币，分别是1, 2, 5，问要组成面值为10的货币有多少种方案。

→ 背包方案总数

该题类似于完全背包问题。但是不同的地方在于他是求刚好装满背包的方案数，而不是求最大价值。

其实，思考方法和前面一样， $f[i]$ 表示凑齐 i 块钱的方案数，那么 $f[i]$ 的方案数就等于 $f[i-a[k]]$ 的方案数之和。 $a[k]$ 表示第 k 种钱币的面值。

```
f[0]=1; //初始化
```

```
for(int i=1; i<=n; i++)  
{
```

```
    for(int j=a[i]; j<=m; j++) //完全背包的写法  
    {
```

```
        f[j]+=f[j-a[i]];
```

```
    }
```

```
}
```

也可以用01背包来写，
时间复杂度比较高。



背包问题

背包问题是递推问题的一个衍生，同时也是动态规划的一个雏形。动态规划是信息学竞赛的一个重难点，要想学好动态规划，先打好基础是非常有必要的。同样的，背包问题远不止这九类，还有很多类型的比较复杂的背包问题，这里先不做累述了。如果遇到比较复杂的背包问题，或者没有讲解过的背包问题，先看看能否转换前面我们学习过的简单背包问题来处理。一定要仔细分析问题，想出解决该类问题的状态状态转移方程（当前状态怎么由前一状态到达）

The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two fluctuating lines is visible on the right side.

Thank you!